Performance Optimization of the TSFM Agent in an Industrial Agentic Benchmark

Alisha Vinod av3311, Thomas Ajai tl3444, Jonathan Ang ma4624, Sanjaii Vijayakumar sv2851

I. Introduction

A. Background

Industrial operations increasingly rely on autonomous agents to monitor, maintain and optimize infrastructure. A vital component of the agentic systems is the ability to perform time series analysis like forecasting and anomaly detection to prevent equipment failures. The AssetOpsBench benchmark, developed by IBM Research, provides a framework for evaluating the agents in industrial scenarios. At the core of this benchmark is the Time Series Foundation Model (TSFM) agent, which uses the TinyTimeMixer (TTM) model family to process sensor data.

B. Problem statement

In industrial asset operations, detecting equipment failures requires running time series forecasting and anomaly detection across multiple sensors and failure mode mappings. A chiller can fail due to multiple independent reasons: compressor overheating, condenser fouling, refrigerant valve failure, and others. Each failure mode is predicted by monitoring a different combination of sensors, and ruling out or confirming a failure requires running time series forecasting and anomaly detection across all relevant sensor streams. A single maintenance decision therefore requires invoking time series inference repeatedly, once per sensor-failure mode mapping, before a work order can be generated. This is the normal structure of industrial predictive maintenance workflows, as illustrated in Figure 1.

Despite the efficiency of TTM, the current implementation of the TSFM agent within the Model Context Protocol (MCP) suffers from performance bottlenecks. Initial profiling showed that the system re-initializes the model from disk on every tool call, creating a cold start overhead that compounds across multi-step workflows. Also, the reliance of wrappers like HuggingFace Trainer adds unnecessary data loading latency, while the default use of float32 precision fails to leverage modern hardware acceleration.

II. Literature Review

The introduction of **AssetOpsBench** [1] by Patel et al. (2025) provides the framework for this project. This benchmark addresses the automation of complex industrial life cycle using autonomous AI agents. It highlights that the industrial assets such as data center chillers, generate a vast amount of multi-modal data, needing agents that can reason over real time IoT telemetry. The paper introduces a 'Plan Execute' vs 'Agent-As-Tool' analysis, demonstrating that LLM based

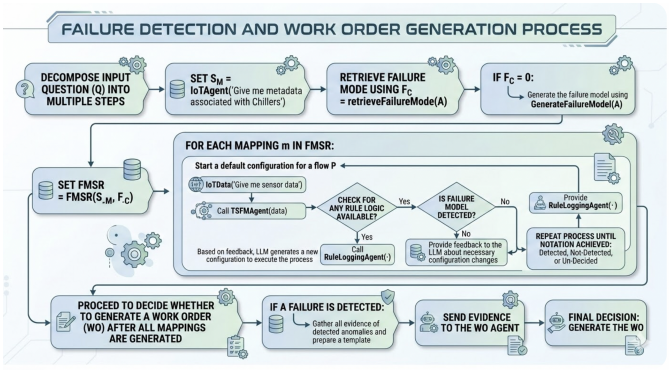

Fig. 1. A single failure detection and work order generation workflow in AssetOpsBench. One workflow calls the IoT agent to fetch sensor data, then invokes the TSFM agent for forecasting or anomaly detection across each sensor-failure mode mapping, and finally triggers the Work Order agent. Reducing TSFM tool call latency therefore has a compounding effect across the entire workflow.

agents struggle with tool orchestration and sequence errors, providing the main motivation for our performance optimizations.

The **TinyTimeMixer (TTM)** [9] architecture represents a significant shift from parameter intensive Transformers to MLP-Mixer based foundation models. TTM is an improvement over the TSMixer architecture, using time-mixing and feature-mixing MLPs shared across time steps to capture information. (OpenReview, 2024). The research indicates that while TTMs achieve state of the art zero shot inferencing in forecasting with only 1M-5M parameters, they remain sensitive to data characteristics like periodicity. This shifted our focus on optimizing the inference pipeline of TTM rather than retraining the model, as its size makes it ideal for resource constrained settings.

AssetOpsBench [1], IBM Research's open benchmark for industrial asset operations, formalizes this structure. It implements four domain-specific agents connected via the Model Context Protocol (MCP): an IoT agent for sensor data retrieval, an FMSR agent for failure mode knowledge, a TSFM agent for time series analysis, and a Work Order (WO) agent for maintenance planning. These agents are orchestrated by a MetaAgent framework in which the LLM issues tool calls iteratively, one at a time, waiting for each result before deciding the next step. The TSFM agent is the computationally heaviest component of this system. It exposes four tasks: time series forecasting (tsfm_forecasting), model fine-tuning (tsfm_forecasting_- tune), anomaly detection (tsfm_integrated_tsad), and

forecasting evaluation.

All tasks are backed by IBM's TinyTimeMixer (TTM) [9], a family of lightweight pretrained Transformer-based time series foundation models. Four TTM variants are available: ttm_96_28 and ttm_512_96 (general-purpose, context lengths 96 and 512), and their energy-domain fine-tuned counterparts ttm_energy_96_28 and ttm_energy_512_96. The benchmark exposes two classes of scenarios that invoke the TSFM agent. The first class consists of direct TSFM scenarios (IDs 216–222) in which the orchestrator issues forecasting or anomaly detection queries directly to the TSFM agent. The second class consists of multi-agent workflow scenarios (IDs 501–520, 621–622) in which the orchestrator first calls the IoT agent to retrieve sensor data from the facility, and then calls the TSFM agent to perform forecasting or anomaly detection on that data before deciding whether to generate a work order. Both classes ultimately invoke the TSFM agent's forecasting or timeseries_anomaly_detection tools, making TSFM tool call latency the shared bottleneck across the entire benchmark.

The performance potential of PyTorch 2's compilation pipeline is limited by graph breaks, which force a fallback to Python's eager mode and cause CPU-GPU synchronization, as stated by Kashmira et al. (2025) [7]. In their study of 195 models, Kashmira et al. (2025) found that 13.8% of models suffer from these breaks. Their work on GraphMend shows that fixing these breaks can lead to 5X average cold start speedup, which reinforces our approach to use *torch.compile* with a focus on eliminating graph breaks and to ensure the model remains in high performance compiled state throughout the tool call. [4]

Recent advancements in **Mixed Precision Quantization** addresses the computational demands of foundation models. Research by Duan et al. (2025) [8] indicates that post-training quantization can convert high precision floating point parameters to low bit integers without the need for expensive retraining. By applying these techniques, especially **bf16**, we can preserve accuracy while reducing memory bandwidth requirements by half, which is vital for multi-sensor scenarios in AssetOpsBench.

III. Methodology

A. Dataset and model

The experimental setup for this study utilizes the TinyTimeMixer (TTM) model, a lightweight time-series foundation model developed by IBM Research. The evaluation focuses on two specific model variants: ttm_96_28, which features approximately 1 million parameters with a context length of 96 and a prediction horizon of 28, and the larger ttm_512_96, which scales to 5 million parameters with a context length of 512 and a horizon of 96. Implementation is supported by the HuggingFace Transformers framework in conjunction with the granite-tsfm library [2].

For the empirical analysis, we employ the IBM AssetOpsBench - Chiller 6 Sensor Data (specifically the chiller6_june2020_sensordata_couchdb file). This dataset consists of 2,896 rows and 12 sensor columns, capturing critical operational metrics such as: Temperature and refrigerant pressure, Efficiency and tonnage, Power input and flow rate.

The dataset is licensed under Apache 2.0. Notably, the methodology relies exclusively on zero-shot inference, meaning no specific train/test splits were created for model fine-tuning.

The computational workload is orchestrated by the IBM Granite 3.3 8B Instruct LLM via the watsonx.ai [3] API. Hardware execution is performed on a GCP n1-standard-4 instance equipped with an NVIDIA T4 GPU and 16GB of VRAM. The software environment is standardized on PyTorch 2.6 and CUDA 12.4 to ensure high-performance processing and compatibility.

B. Optimization Scope

We focus exclusively on inference optimization rather than training, as the TTM model is used in zero-shot mode, no fine-tuning is performed. The optimization scope spans four techniques applied incrementally:

- **Model caching and pre-loading:** In the baseline, every tool invocation calls `TinyTimeMixerForPrediction.from_pretrained()`, incurring a 233ms cold-start penalty per request. We eliminate this by pre-loading all TTM checkpoints at server startup and storing references in a module-level cache, ensuring subsequent calls retrieve the loaded model.
- **Graph compilation with `torch.compile`:** The baseline eager-mode execution dispatches 19,032 individual CUDA kernel launches per forward pass, each carrying fixed launch overhead. We apply `torch.compile(mode='reduce-overhead')` to fuse adjacent operations into fewer, larger kernels.
- **Trainer replacement with direct model calls:** The baseline wraps inference in a HuggingFace `Trainer.predict()` call, which introduces a DataLoader iteration loop. We replace this with a single direct `model(**batched_inputs)` call that processes the entire input in one shot, eliminating all DataLoader and per-batch copy costs.
- **GPU placement and mixed-precision inference.** The baseline runs entirely on CPU despite GPU availability. We move the model to CUDA via `model.cuda()` and enable TF32 matrix multiplication precision via `torch.set_float32_matmul_precision('high')` to leverage Tensor Core acceleration on the T4 GPU. We additionally evaluate bf16 mixed precision via `torch.autocast(device_type='cuda', dtype=torch.bfloat16)` to assess whether halved memory bandwidth yields further speedup for TTM's MLP-based architecture.

HuggingFace trainer does not need to be initialized and we also remove the overhead by using the model directly

Listing 1 Refactoring of the inference engine in `forecasting.py`. Preload and caching of bf16 model in `model_cache.py`.

```
@@ -275,11 +292,11 @@ def _get_ttm_hf_inference(
- model = TinyTimeMixerForPrediction.from_pretrained(...)
+ model = get_compiled_model("ttm_96_28")
+ if model is None:
+     raise RuntimeError(f"Model not pre-loaded")
```

```
@@ preload_and_compile_models(
+ model.to(device="cuda", dtype=torch.bfloat16)
+ compiled_model = torch.compile(model,
↪  mode='reduce-overhead')
+ # Store compiled model
+ _COMPILED_MODELS[model_name] = compiled_model
```

C. Profiling tools

We used a combination of lightweight wall-clock profiling, PyTorch operator-level profiling, and experiment tracking through Weights & Biases [6]. The MCP workflow has overhead from the agent call, data loading, model setup, inference, and result serialization, while PyTorch profiling only explains what happens inside the model execution path. Hence we used multiple profiling tools to get the best possible full picture.

a) Custom TSFM profiler.: We added a small profiler around the main TSFM forecasting path to measure the latency of each stage in the MCP tool call. This profiler records wall-clock time for data loading, preprocessing, model loading, `ttm_forward`, and postprocessing stages. This was the most useful tool for understanding end-to-end workflow latency because it measures the system the same way the user experiences it: from the agent request to the saved forecast result. It also helped us separate actual model inference time from surrounding pipeline overhead.

b) PyTorch Profiler.: For lower-level analysis, we used `torch.profiler.profile` with both CPU and CUDA activities enabled. This gave us an operator-level view of the inference call, including CPU-side compilation overhead, matrix multiplication calls, CUDA kernel activity, and host-to-device memory copies. This helped us understand the effect of `torch.compile`. The profiler trace showed that the first compiled call is dominated by compilation and graph setup, so the compiled model only helps if the model is reused across later MCP tool calls, which offsets the cost of compilation. It also showed that BF16 kernels were active, and also that the workload was not purely GPU-compute-bound.

c) Weights & Biases tracking.: We used Weights & Biases to track the results of our sweep runs across different sensor counts and optimization settings. For each run, we logged the total workflow latency, `ttm_forward` latency, model loading time, data loading time, and peak VRAM usage. It helped us see the optimization behavior changes as we moved from 1 sensor to 2, 5, and 9 sensors.

d) Sweep script.: We wrote a sweep script to run the same `plan-execute` workflow repeatedly using different sensor sets. Each configuration used one warmup run followed by three measured runs. The script parsed the profiler output from the terminal logs and averaged the measured values before logging them to Weights & Biases. This kept the comparison consistent across configurations and reduced the chance of over-interpreting a single noisy run.

Overall, the profiling setup gave us two levels of evidence. The custom profiler and Weights & Biases showed where time was spent in the full MCP workflow, while PyTorch Profiler explained what was happening inside the model execution path. This combination was important because the largest bottleneck was not only model compute. A large part of the observed latency came from repeated setup, compilation warmup, and framework overhead around the actual TTM forward pass.

D. Evaluation metrics

We evaluate performance along three axes: latency, memory, and scalability.

- **Workflow completion latency:** The total wall-clock time from querying to forecast result, measured in milliseconds using `time.perf_counter()`. We have decomposed the pipeline into five per-stage measurements: data loading, preprocessing, model loading, `ttm_forward`, and postprocessing and measured the per-stage latency using our custom TSFM profiler. We discard one cold run to absorb the `torch.compile` JIT cost and average 3 subsequent warm runs per configuration.
- **Peak GPU memory:** Peak VRAM per stage recorded in MB using `torch.cuda.max_memory_allocated()`, reset between stages.
- **Sensor count scalability:** We evaluated the workflow across {1, 2, 5, 9} sensors, mirroring the AssetOpsBench scenario 216. This shows whether optimizations are dependent on sensor count or degrade with multivariate input size.
- **Speedup ratio:** We measure the ratio between baseline latency and optimized latency for both `ttm_forward` and total latency. Ablation speedup isolates the incremental contribution of each optimization over the previous stack.
- **Kernel-level profiling:** We use `torch.profiler.profile` with CPU and CUDA activities to inspect the operator-level behavior of the inference call. These metrics which include CPU-side compilation overhead, matrix multiplication calls, CUDA kernel activity, tensor copy operations, and host-to-device memory transfers are mainly used for interpretation and not as the primary end-to-end metric.
- **Cold-start and warm-run latency:** We separate the first run from the measured warm runs because `torch.compile` adds a one-time graph compilation cost. The warm runs better represent the expected latency when the TSFM MCP server is already initialized and the compiled model is available in memory.
- **Run-to-run variability:** For each configuration, we run one warmup call followed by three measured calls. The average latency across the measured runs are reported to reduce the effect of one-off noise from server startup,

TABLE I
PER-STAGE LATENCY: BASELINE VS. OPTIMIZED (WARM RUN, 3 SENSORS)

Stage	Baseline (ms)	Optimized (ms)	Δ
Data loading	128	131	∼0%
Preprocessing	45	35	−22%
Model loading	233	26	−89%
`ttm_forward`	40,161	12,293	−69%
Postprocessing	848	769	−9%
Total	**43,021**	**13,798**	**−68%**

Python overhead, CUDA initialization, and background system activity.

- **Forecast validity:** We also check that each optimized run completes successfully and produces a forecast output file with the expected target columns and forecast horizon. This is a basic check to make sure the optimized path is still executing the same forecasting workflow.

IV. EXPERIMENTAL RESULTS

A. Baseline Profiling

We first ran the IBM codebase for TTM inference on the CPU via HuggingFace Trainer, reloading the model from disk on every tool call. The results are as follows: baseline `ttm_forward` is 40,161ms, total latency 43,021ms, and model loading 233ms per call. They also showed `torch.profiler` 19,032 CUDA kernel launches per forward pass and 458ms of DataLoader overhead from per-batch memory copies which then became teh primary optimization targets.

B. Per-Stage Breakdown

Table I compares per-stage latency between the baseline and fully optimized configuration.

C. Ablation results

Table II shows the incremental contribution of each optimization.

D. Key Findings

- The model inference time ttm_forward dominates. At 93% of total latency, all optimization effort is targeted at the forward pass.
- **`torch.compile` is the dominant optimization.** Eliminating 19,032 CUDA kernel launches per forward pass via kernel fusion accounts for the largest single speedup, reducing `ttm_forward` from 40,161ms to 16,000ms.
- **Model pre-loading reduces cold-start overhead by 9×.** Loading time drops from 233ms to 26ms per tool call and remains constant across all sensor counts.
- **GPU placement adds 1.3× on top of `torch.compile`.** Moving the model to CUDA with TF32 matrix multiplication precision reduces `ttm_forward` from 16,000ms to 12,293ms.
- **bf16 reduces performance.** TTM's MLP architecture with small matrix dimensions does not benefit from Tensor Core acceleration; bf16 conversion overhead outweighs compute savings, increasing `ttm_forward` to 15,272ms.
- **CPU baseline is by design.** TTM is an MLP-based model with no attention mechanism, explicitly designed by IBM for efficient CPU-only deployment in resource-constrained industrial environments [2]. The baseline therefore runs on CPU by design rather than misconfiguration. GPU placement was introduced as an additional optimization to leverage Tensor Core acceleration and enable bf16 mixed-precision evaluation.

V. DISCUSSION

A. Interpretation

What delivered significant speedups was using torch.compile(). This is because Pytorch's TorchInductor is able to do kernel fusion optimizations that eliminated 19,032 CUDA launches by fusing into a single kernel. Secondly, pre-loading and caching of the TTM model also reduced loading time from 230ms to 25ms, 9x speedup per tool call.

The use of mixed precision hurt performance rather than increase performance which we expected. This shows that the conversion cost from tf32 to bf16 was more than the speedups of running tensor core operations in bf16. It could be possible that TTM's small matrix dimensions as a MLP-based, no attention architecture does not saturate Tensor Cores, so the dtype conversion overhead dominates.

The next speedup was the use of GPU which delivered 1.3x speedups. However, this pales in comaprison to the other speedups, pointing to the main bottleneck of the MCP server being the latency of each MCP tool call.

B. Limitations

Since we used a single dataset, Chiller 6, June 2020, our results may not generalize to all AssetOpsBench scenarios or other industrial sensor profiles.

The speedups investigated by GPU placement only reflected that of a single GPU instance and may differ on newer hardware (A100, H100) where bf16 Tensor Cores are more mature. Additionally, the weaker than expected speedups from using GPUs could be attributed to TTM being more CPU-optimized by IBM.

Another limitation in this investigation is that the 3-run average after one warmup does not reflect a cold-start scenario where torch.compile's cost are higher due to JIT.

C. Future work

To address the limited scope of the dataset used, a followup would be to use larger sensor sets and longer context variants (ttm_512_96) to see whether GPU gains scale with input size. We could also evaluate on full AssetOpsBench official scenarios (IDs 216–222, 501–520) rather than the custom sweep.

TABLE II
ABLATION STUDY: INCREMENTAL SPEEDUP PER OPTIMIZATION

Configuration	ttm_forward (ms)	Total (ms)	Speedup
Baseline CPU	40,161	43,021	1.0×
+ Pre-loading + torch.compile + no Trainer	16,000	17,000	2.5×
+ GPU placement	12,293	13,798	3.3×
+ bf16 autocast	15,272	16,414	2.6×

Another optimization that could be made is creating a persistent MCP server through socket reuse to eliminate repeated server startup costs. Furthermore, we could explore batching multiple sensor streams into a single model call rather than one call per sensor-failure pair. Lastly, we could explore INT8/INT4 post-training quantization that we expect would further reduce memory bandwidth and produce some speedups.

VI. CONCLUSION

In this project, we optimized the TSFM MCP forecasting path in AssetOpsBench with a focus on reducing workflow latency rather than changing the forecasting model itself. We started by profiling the full workflow end-to-end. The MCP tool call included data loading, data quality filtering, preprocessing, model setup, inference, and postprocessing. Splitting the pipeline into these stages made it much easier to see which parts were actually worth optimizing.

Our main improvements came from model preloading, model caching, `torch.compile`, and replacing the HuggingFace `Trainer` inference path with a direct batched model call. These changes reduced repeated setup overhead and made the inference path simpler. We also tested BF16 mixed precision, but it did not give a large speedup for this workload. The BF16 kernels were active, but the trace still showed CPU-side compilation, framework overhead, and tensor conversion costs, these insights were driven from the PyTorch profiler.

The biggest lesson from the project is that optimizing an agentic ML system is different from optimizing only a model. In a standalone script, model inference might be the only thing we care about. But in AssetOpsBench, however, the end-to-end latency also depends on how the MCP server is started, how often the model is loaded, how the planner calls the tool, and how the results are serialized. For this reason, the most useful measurements were the ones that captured the full workflow instead of only the GPU kernels.

There are still some limitations in our current setup. Our experiments were run on the Chiller 6 sample dataset and a single GPU instance, so the results should be treated as a focused benchmark rather than a complete evaluation of all AssetOpsBench scenarios. The current setup also still has server startup and orchestration overhead that could be reduced further with a persistent MCP server. As future work, we would extend the same profiling setup to more official AssetOpsBench scenarios, test larger datasets and model variants, and explore quantization once the CUDA environment is stable.

Overall, the project shows that careful profiling can lead to practical performance improvements even without changing model weights. For the TSFM MCP agent, the most effective gains were not just from moving the model to the GPU or simply due to switching to BF16, but to remove repeated setup costs and reuse the compiled forecasting model across the tool calls.

REFERENCES

[1] D. Patel, S. Lin, J. Rayfield, N. Zhou, R. Vaculin, N. Martinez, F. O'Donncha, and J. Kalagnanam, "AssetOpsBench: Benchmarking AI Agents for Task Automation in Industrial Asset Operations and Maintenance," *arXiv preprint arXiv:2506.03828*, 2025.

[2] IBM Research, "Granite-TSFM: Industrial Strength Time Series using Foundation Models," GitHub repository, 2024. https://github.com/ibm-granite/granite-tsfm

[3] IBM, "IBM watsonx.ai: Foundation Model Platform," 2024. https://www.ibm.com/products/watsonx-ai

[4] PyTorch Team, "Introducing TorchDynamo: A Python-Level JIT Compiler for PyTorch," 2022. https://pytorch.org/docs/stable/torch.compiler.html

[5] PyTorch Team, "PyTorch Profiler," 2023. https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html

[6] Weights & Biases, "Weights & Biases: The AI Developer Platform," 2020. https://wandb.ai

[7] S. Kashmira, J. Dantanarayana, T. Sathiyalogeswaran, Y. Yuan, N. Talati, K. Flautner, L. Tang, and J. Mars, "GraphMend: Code Transformations for Fixing Graph Breaks in PyTorch 2," *arXiv preprint arXiv:2509.16248*, 2025.

[8] Anonymous Authors, "Mixed-Precision Quantization for High Fidelity Segmentation in Resource Constrained Scenarios," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 39, 2025.

[9] Anonymous Authors, "Tiny Time Mixers: Fine-Tuned MLP-Mixer Foundation Models as data-driven Numerical Surrogates?," *OpenReview*, 2024. https://openreview.net/pdf?id=dUg1dihFDT